

Category 8: Security Audit

What you are measuring: The real attack surface of a production government web application. Ship has authentication, WebSocket connections, user-generated content, a PostgreSQL backend, and third-party dependencies. Your job is not to run a scanner and report the output. Your job is to build a tool that actively probes the running application, then interpret what it finds.

The Security Probe Tool

You must build a security probe tool as part of this category. The tool is a deliverable, not optional. It must be a runnable script or CLI that actively tests the live application across at least four attack surfaces:

- Authentication and session handling: test for weak session tokens, missing token expiry, unauthenticated route access, and privilege escalation between user roles
- WebSocket message validation: send malformed, oversized, and unexpected message types to the WebSocket endpoint. Does the server crash, silently accept invalid state, or reject correctly?
- Input sanitization: test for XSS, SQL injection, and excessively long input across all user-facing fields. Include both stored and reflected vectors
- Dependency vulnerabilities: programmatically run npm audit and parse the output. Flag any high or critical CVEs and identify which application features depend on the vulnerable package

The tool must produce a structured report (JSON or markdown) with findings, severity ratings, and reproduction steps for each issue. It must be runnable by a grader against a fresh instance of the app with a single command.

Manual Review Requirements

In addition to the probe tool, conduct manual review of:

- CORS and CSP configuration: are cross-origin requests properly restricted?
- Environment variable and secret handling: are secrets ever exposed to the client bundle or logged?
- Rate limiting: can a single client hammer the API or WebSocket endpoint without restriction?
- Error message verbosity: do error responses leak stack traces, SQL, or internal paths?

Audit Deliverable

Metric	Your Baseline
Security probe tool	Runnable (Yes / No)
Auth/session vulnerabilities found	List with severity
WebSocket validation failures	List with severity
Input sanitization failures	List with severity

High/Critical CVEs in dependencies	Count + list
CORS/CSP misconfiguration	Yes / No + details
Secrets exposure risk	Yes / No + details
Rate limiting absent on endpoints	List
Verbose error leakage	Yes / No + examples

Improvement Target

Fix at least 2 verified vulnerabilities with before/after proof. Each fix must include: the vulnerability class, the reproduction steps used to confirm it, the fix applied, and evidence that the fix works (tool output before vs. after, or a test that catches the regression). Fixes must not break existing tests.